Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 80%. We keep your highest score.

Next Item →

Agent architecture design

Overview

The agent's complete tool architecture



In this activity, you will build the core intelligence for an AI research assistant. This activity brings together all the skills you have learned, focusing on function calling—the ability of the AI to choose the right tool at the right time.

Your primary goal is to architect and define custom functions and integrate them seamlessly with built-in Gemini tools (Google Search and URL Context) to handle complex, multi-step user requests within a conversation loop.

Scenario

You are tasked with building a prototype for an AI research assistant. This assistant must be able to search the web for recent news, save important notes to a file, and summarize information for the research team, operating smoothly within a single conversation.

Steps

Step 1: Setup and secure the agent

We begin by setting up the secure project environment and the base code that handles client initialization.

1. **Create project and secure key:** Create a new project folder. Create a `.env` file for your API key and a `.gitignore` file to exclude `.env` (security best practice).
2. **Install libraries:** Install the necessary packages.
   - **Python:** `pip install google python-dotenv`
3. **Write starter code:** Copy the following code into your main file.

(Python starter code)

```python
import os
from typing import Any, Dict, List

from dotenv import load_dotenv
from google import genai
from google.genai import types

# -- Setup & client ------------------------------------------------------------
load_dotenv()
api_key = os.getenv("GEMINI_API_KEY")
if not api_key:
    raise RuntimeError("GEMINI_API_KEY is not set")

client = genai.Client(api_key=api_key)
# Default to Gemini 3 Flash preview (override with GEMINI_MODEL="gemini-3-pro-preview" if desired)
MODEL_ID = os.getenv("GEMINI_MODEL", "gemini-3-flash-preview")
```

1. You are setting up the project for your AI research assistant. You have your Gemini API key and need to make it available to your Python script without exposing it in the code, as you plan to share your project on GitHub. Which of the following is the most secure and standard practice? Select the best answer. 1 / 1 point

   - (selected) Create a `.env` file to store the key, add `.env` to your `.gitignore` file, and use a library to load the key as an environment variable in your script.
   - Create a separate Python file named `secrets.py`, store the key in a variable there, and import it into your main script.
   - Paste the API key directly into your main script as a string variable.
   - Encrypt the key and include the encrypted string and the decryption password in the code.

   Nice work

   Correct: This method keeps your secret key out of your source code and prevents it from being accidentally committed to version control, which is a critical security practice.

Step 2: Implement custom functions and schemas

You must implement the definitions for the three functions the agent will use.

Action:

1. **Analyze the `save_note` function** (the actual Python function with error handling) provided as an example.
2. **Analyze the schema** (function declaration) for `save_note`. This is given as an example.

Define stubs/schemas for the other two required functions (`analyze_sentiment`, `generate_concise_summary`).

```python
# -- 1. Agent Function Definition (Example: save_note) ---
def save_note(content: str, filename: str) -> str:
    """Append a note to a local text file."""
    try:
        with open(filename, "a", encoding="utf-8") as f:
            f.write(content + "\n\n")
        return f"Note saved to '{filename}'."
    except Exception as e:
        return f"Error saving note: {e!r}"

def web_research(query: str, url: str | None = None) -> Dict[str, Any]:
    """
    Use Google Search + URL Context to get fresh info and summarize it.
    This is where we integrate the google_search + url_context tools.
    """
    tools = [
        {"google_search": {}},
        {"url_context": {}},
    ]

    prompt_parts = [
        f"Use Google Search to find recent, relevant information about: {query}.",
        "Provide a short 3-bullet summary with one sentence per bullet.",
    ]
    if url:
        prompt_parts.append(
            f"Also read and use this URL as context if relevant: {url}"
        )

    prompt = " ".join(prompt_parts)

    try:
        resp = client.models.generate_content(
            model=MODEL_ID,
            contents=prompt,
            config=types.GenerateContentConfig(tools=tools),
        )
        summary_text = resp.text or ""
        return {"status": "ok", "summary": summary_text}
    except Exception as e:
```

2. You have defined a function `save_note(content: str, filename: str)` for your AI agent. You provide a prompt: "Please save the key takeaways from our meeting into a file called meeting_notes.txt." However, the model does not call your function and instead just replies with text. What is the most likely reason the function was not called? Select the best answer. 1 / 1 point

   - (selected) The function's description in the schema was unclear or did not accurately explain that it saves content to a file.
   - You forgot to include a `return` statement in your `save_note` function definition.
   - The model is not capable of writing to files, so it ignored the function.
   - The function needs to be defined using the `async def` syntax to work with the model.

   Nice work

   Correct: The model relies heavily on the function's name and, more importantly, its description to understand its purpose and when to use it. A clear, descriptive explanation is crucial.

3. When defining the `parameters_json_schema` for a custom function, you decide to mark a field like `filename` as required. What is the most important reason for explicitly listing a parameter as required in the schema? Select the best answer. 1 / 1 point

   - It automatically creates a new folder on the developer's computer to store the notes.
   - It makes the local Python code run significantly faster when opening the file.
   - It allows the model to ignore the user's instructions if it thinks the filename is not important.
   - (selected) It forces the model to always identify and provide a value for that parameter before it attempts to call the function.

   Nice work

   Correct: If a parameter is required, the model knows it cannot proceed with a tool call unless it has gathered or generated that specific piece of information.

Step 3: Architecting the orchestration

This section handles any tool/function requests returned by the model, runs the appropriate local logic with the provided inputs, and produces a structured result. That result is then packaged in the correct format so it can be added back into the conversation for the model to use in a follow-up response. It also includes a small helper to consistently extract readable text from model outputs.

Action 1: Define all tools: Fix the code below with the missing text to create a single tool configuration object that includes your three custom functions and the web search built-in tool.

```python
# -- Function-call dispatcher --------------------------------------------------
def run_custom_function(fc: types.FunctionCall | Any) -> Dict[str, Any]:
    name = getattr(fc, "name", None)
    call_log = getattr(fc, "function_call", fc)
    args = dict(getattr(call_log, "args", {}) or {})

    try:
        if name == "<TYPE CODE HERE>":
            msg = save_note(args["content"], args["filename"])
            return {"status": "ok", "message": msg}

        if name == "<TYPE CODE HERE>":
            analysis = analyze_sentiment(args["text"])
            return {"status": "ok", "analysis": analysis}

        if name == "<TYPE CODE HERE>":
            summary = generate_concise_summary(
                args["text_input"],
                int(args["max_words"])
            )
            return {"status": "ok", "summary": summary}

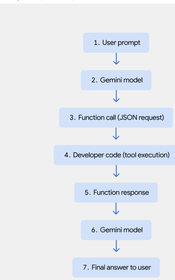
        if name == "<TYPE CODE HERE>":
            return web_research(
                query=args["query"],
                url=args.get("url"),
            )

        return {"status": "error", "error": f"Unknown function '{name}'."}

    except Exception as e:
        return {"status": "error", "error": str(e)}

# -- Response text helper ------------------------------------------------------
def extract_text(response: types.GenerateContentResponse) -> str:
    if getattr(response, "text", None):
        return response.text
    pieces = []
    for cand in response.candidates or []:
        if cand.content and cand.content.parts:
```

The function calling workflow (the two-turn loop)



Action 2: Implement the two-turn conversation loop: Implement the main conversation loop that handles function calls (the model's request to run your Python code) and then sends the result back to the model for a final, user-friendly response.

- **The manager/employee analogy:** The function calling mechanism works in two turns. *The AI* (the manager) asks your code (the employee) to run a tool (turn 1), and then your code sends the results back (turn 2). This prevents the AI from freezing while waiting for slow external tasks.
- **Fix [#TYPE CODE HERE]** the code in the "second half" of the function-calling loop to take the model's requested function calls, run them in Python, and package the results back into the chat history so the model can see them.
- **Run the script** and interact with the AI by testing the different tools.

```python
# -- Two-turn function-calling per user prompt ---------------------------------
def handle_user_turn(history: list[types.Content], user_prompt: str) -> str:
    # 1) User message
    user_content = types.Content(
        role="user",
        parts=[types.Part.from_text(text=user_prompt)],
    )
    history.append(user_content)

    # First turn: model decides which function(s) to call
    try:
        first = client.models.generate_content(
            model=MODEL_ID,
            contents=history,
            config=types.GenerateContentConfig(
                tools=TOOLS,
# Use AutomaticFunctionCallingConfig, a google-genai SDK configuration object that controls whether the model is allowed to automatically invoke declared functions/tools during generate_content().
                automatic_function_calling=types.AutomaticFunctionCallingConfig(
                    disable=True  # we manage function calling manually
                ),
            ),
        )
    except Exception as e:
        return f"Error calling the model: {e!r}"

    history.append(first.candidates[0].content)
    function_calls = first.function_calls or []

    # Model answered directly (or only used its own tools inside web_research)
    if not function_calls:
        return extract_text(first)

    # 2) Execute requested custom functions and send results back

    # TYPE CODE HERE

    # Second turn: final answer
    try:
        second = client.models.generate_content(
```

4. Your AI agent has received a function call from Gemini to run `web_research`. In your two-turn function-calling loop, you execute the function like this: 1 / 1 point

   Code:

   ```
   result = run_custom_function(fc)
   ```

   To complete the two-turn workflow in this codebase so Gemini can use the tool output when generating the final response, what should you do next? Select the best answer.

   - Print result and end the turn, since tool output is already the final answer.
   - (selected) Wrap result with types.Part.from_function_response(...), append it as types.Content(role="tool", parts=...) to history, then call generate_content() again.
   - Replace the tool result with a natural-language summary and add it as a role="user" message.
   - Call client.models.generate_content() again using the same history without appending anything new.

   Nice work

   Correct: That's exactly what your code does with tool_parts and history.append(types.Content(role="tool", parts=tool_parts)), enabling Gemini to read the tool output on the second turn and produce the final answer.

5. In a research assistant application, the agent determines it needs to call a function. The developer's code executes the local Python logic and gets a result. Why must this result be appended back to the `history` as a "tool" role before calling the model a second time? Select the best answer. 1 / 1 point

   - (selected) To provide the model with the missing information it requested, allowing it to complete the user's original task
   - To erase the user's initial prompt and start a brand new conversation from scratch
   - To change the model's personality to be more professional and concise
   - To notify the user that the code has finished running so they can close the terminal

   Nice work

   Correct: The "tool" role provides the context of the function's output. Without this step, the model would be "blind" to the results of the work it asked the code to perform.

Final checklist

Your AI agent architecture is complete when:

1. The three custom functions have clear function definitions and schema declarations.
2. The **`AGENT_CAPABILITIES`** list correctly includes your three custom functions plus the Google Search and URL Context tools.
3. The two-turn conversation loop correctly handles the function call request, executes the function, and sends the result back to the model.

Next steps

The next step is to test your agent by providing a user prompt that requires it to use at least three tools sequentially. Your agent should respond by chaining those tools/functions together to achieve the final output.

Key points

- **Function declarations:** The most critical step in function calling is writing a clear, accurate description (the `description` field) for each function. The AI relies entirely on this text to decide whether or not to call your code.
- **Agent orchestration:** You do not need to manually code the sequence (Search → Summarize → Save). By providing the Gemini model with access to all the tools and functions simultaneously, its intelligence handles the complex, multi-step orchestration automatically.

Exemplar solution (Optional)

(The Exemplar shows the required final logic for the definitions and the execution map.)

```python
import os
from typing import Any, Dict, List

from dotenv import load_dotenv
from google import genai
from google.genai import types

# -- Setup & client ------------------------------------------------------------
load_dotenv()
api_key = os.getenv("GEMINI_API_KEY")
if not api_key:
    raise RuntimeError("GEMINI_API_KEY is not set")

client = genai.Client(api_key=api_key)

# Default to Gemini 3 Flash preview (override with GEMINI_MODEL="gemini-3-pro-preview" if desired)
MODEL_ID = os.getenv("GEMINI_MODEL", "gemini-3-flash-preview")

# -- Custom functions (business logic) -----------------------------------------
def save_note(content: str, filename: str) -> str:
    """Append a note to a local text file."""
    try:
        with open(filename, "a", encoding="utf-8") as f:
            f.write(content + "\n\n")
        return f"Note saved to '{filename}'."
    except Exception as e:
        return f"Error saving note: {e!r}"

def analyze_sentiment(text: str) -> Dict[str, Any]:
    """Very simple rule-based sentiment analysis."""
    text_lower = text.lower()
    pos = ["good", "great", "excellent", "positive", "happy", "love"]
    neg = ["bad", "terrible", "awful", "negative", "sad", "hate"]

    score = sum(w in text_lower for w in pos) - sum(w in text_lower for w in neg)
    label = "Positive" if score > 0 else "Negative" if score < 0 else "Neutral"
    return {"sentiment": label, "score": score}
```

How does your work compare?

Use these questions to evaluate the quality and robustness of your final agent architecture:

- **Function clarity:** Did you write a clear, descriptive explanation (schema description) for all three custom functions? (The model relies entirely on this text to decide when to call your code.)
- **Complete toolset:** Did you successfully configure the API to enable access to all five required tools (your three custom functions + Google Search + URL Context)?
- **Execution flow (two-turn):** Does your conversation loop correctly implement the two-turn process? (i.e., Did you check for the function call, execute the function, and then send the result back to the model for the final response?)
- **Robustness:** If one of your functions were slow (like a complex database query), is your application defined with `async` to remain responsive and avoid freezing?
- **Local error handling:** Did you wrap your local function logic (like file writing) in a **`try...except` block** to return a friendly error message to the model if the operation fails?